

AI 시스템 디자인 패턴

서비스 구성과 배포



8가지 AI 설계 문제 (Application-side + AI System-side)

1 AI 서비스 조합 여러 AI Task 연결 구조	2 AI 서비스 결정 모델/엔드포인트 선택
3 AI 서비스 배포 물리적 위치 결정	4 컨텍스트 정확성 상태/기억 유지
5 컨텍스트 효율성 데이터 공급 최적화	6 신뢰성 확보 가드레일/검증
7 수행 효율성 처리량/응답성	8 진화 지원 모델 교체/학습

Part 1 집중 영역

1-4번 문제: 서비스 조합, 결정, 배포, 컨텍스트 정확성
핵심: 구조적 기반 설계

핵심 질문

- ? 어떻게 AI 태스크를 조립할 것인가?
- ? 어떤 모델/엔드포인트를 언제 호출할 것인가?
- ? 어디에 배포할 것인가(엣지/서버/하이브리드)?
- ? 대화/세션 맥락을 어떻게 정확히 유지할 것인가?



AI 서비스 조합 (AI Service Composition)

실제 AI 시스템은 단일 모델이 아닌, 여러 독립적인 AI 기능이 협력하여 하나의 비즈니스 서비스를 완성하는 구조가 일반적입니다. 단순한 기능 나열을 넘어 제어 방식과 유연성을 설계해야 합니다.

💡 실제 예시: 기업 문서 기반 질의응답 챗봇



"작년 정보보호 감사 결과를 요약해 주세요"



3가지 대표 조합 패턴



1-1. 정적 선형 파이프라인 Static Linear

고정된 순서로 직렬 실행하여 단순하고 예측 가능성이 높음.

제어 방식: 고정

유연성: 낮음



1-2. 중앙 집중형 오케스트레이션 Orchestration

중앙 라우터가 조건에 따라 다음 태스크를 동적으로 결정.

제어 방식: 중앙 집중

유연성: 중간



1-3. 자율적 에이전트 루프 Agentic Loop

AI가 스스로 계획을 수립하고 도구를 선택하며 반복 탐색.

제어 방식: 자율적

유연성: 높음



핵심 질문: 여러 독립적인 AI 태스크를 어떤 구조로 연결할 것인가?

설계 고려사항

데이터 흐름

실행 순서

제어 방식

실패 처리

유연성 수준



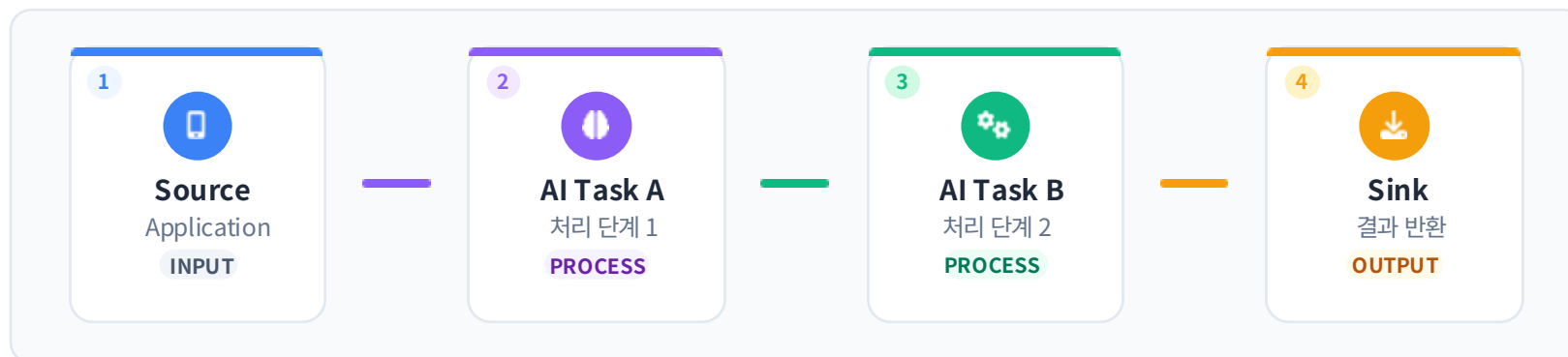
정적 선형 파이프라인 패턴 (Static Linear Pipeline Pattern)

비즈니스 로직을 고정된 순서의 데이터 처리 흐름으로 정의하여 예측 가능성을 높이고 불필요한 제어 오버헤드를 제거하는 전략입니다.



컴포넌트 및 커넥터 구조

Component and Connector View



1 단방향 데이터 파이프

각 단계의 출력이 다음 단계의 입력이 되는 직렬 구조로, 데이터 흐름이 고정되어 높은 예측 가능성을 보장합니다.

#예측가능성

2 순차 실행 (Sequential)

AI Task A → AI Task B 순으로 순차적으로 실행되며, 병렬 처리나 조건부 분기 없이 명확한 흐름을 가집니다.

#구조단순성

3 제어 오버헤드 최소화

중앙 관리자 없이 태스크 간 직접 연결되어 제어 로직이 단순하고 응답 지연이 최소화됩니다.

#저지연처리

커넥터 정보

Unidirectional Data Pipe

단방향 데이터 스트림

Sequential API Call

순차적 API 호출

주요 특징

- 구조 단순성으로 인한 쉬운 설계
- 고정된 흐름으로 높은 예측 가능성
- 동적 분기가 없어 저지연 처리 가능
- 단순한 실행 경로로 디버깅 용이



정적 선형 파이프라인 패턴 평가

고정된 순서의 직렬 구조를 통해 예측 가능성을 높이고 제어 오버헤드를 제거한 패턴의 실무 적용성과 주의사항을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **저지연 처리:** 중앙 관리자 없이 직접 연결되어 응답 속도가 빠릅니다.
- ✓ **단순성 및 디버깅 용이:** 구조가 직관적이며 디버깅이 쉽습니다.
- ✓ **예측 가능성:** 경로가 고정되어 실행 결과를 미리 예측할 수 있습니다.



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **유연성 낮음:** 단계 추가나 조건 분기 변경 시 전체 구조 수정이 필요합니다.
- ⚠ **변경 비용 높음:** 엔드포인트 변경 시 코드 수정 및 재배포가 요구됩니다.
- ⚠ **장애 전파:** 특정 단계에서 실패하면 전체 파이프라인이 중단됩니다.



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **고정 흐름 파이프라인:** 데이터 흐름이 고정적이며 변경이 거의 없는 환경
- ✓ **초저지연 성능:** 실시간 객체 탐지나 단일 목적의 텍스트 분류 등
- ✓ **단순 데이터 처리:** 순차적인 데이터 변환과 파싱이 필요한 경우



주의점 (Considerations) 도입 시 고려사항

2개

- ⓘ **실패 처리/재시도:** 별도의 에러 처리 및 재시도 로직을 견고하게 설계.
- ⓘ **모니터링:** 각 단계의 성능을 모니터링하여 전체 흐름의 병목 현상을 파악



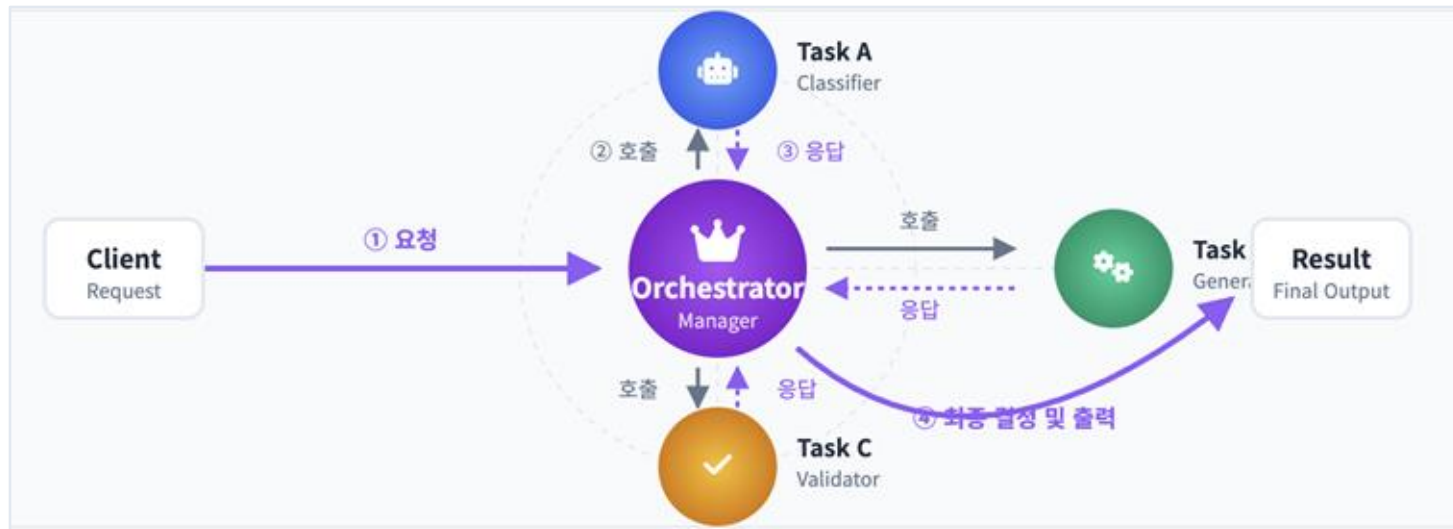
중앙 집중형 오케스트레이션 패턴 (Centralized Orchestration Pattern)

중앙의 관리자가 각 AI 태스크의 실행을 제어하고 결과에 따라 다음 태스크를 동적으로 결정하며, 복잡한 비즈니스 로직을 한곳에서 관리하는 패턴입니다.



컴포넌트 및 커넥터 구조

Component and Connector View



1 스타 토폴로지

Orchestrator가 중앙에서 모든 Worker AI Services와 직접 통신하는 성형(Star) 구조입니다.

2 요청-응답 커넥터

Task를 호출하고 결과를 받은 후, 그 결과값에 따라 다음 Task를 동적으로 결정합니다.

3 중앙 집중 제어

모든 제어권이 중앙 매니저를 거치므로 복잡한 비즈니스 로직 구현에 유리합니다.

i 커넥터 정보

Request-Response Connector

요청-응답 연결

Star Topology

성형 토폴로지

✓ 주요 특징

- 제어 가능성
- 복잡 로직 처리
- 서비스 재사용성
- 에러 처리/재시도



1-2 중앙 집중형 오케스트레이션 - 평가/적용

장점·단점·적용·주의점 분석



중앙 집중형 오케스트레이션 패턴 평가

중앙 관리자가 전체 워크플로우를 제어하고 복잡한 비즈니스 로직을 한곳에서 관리하는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **제어 가능성:** 조건부 분기, 에러 처리, 재시도를 통합 관리합니다.
- ✓ **재사용성:** 개별 서비스가 독립적으로 존재하며 필요 시 자유롭게 조합 가능합니다.
- ✓ **복잡 로직 통합 관리:** 비즈니스 규칙 변경 시 중앙 오케스트레이터만 수정하면 됩니다.



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **병목 현상:** 모든 요청이 중앙 매니저를 거쳐야 하므로 트래픽 집중 시 병목이 발생합니다.
- ⚠ **단일 장애점(SPOF):** 오케스트레이터 장애 시 전체 시스템 서비스가 중단될 위험이 있습니다.
- ⚠ **지연 증가:** 서비스 간 직접 통신이 불가능하고 항상 중앙을 경유하므로 응답 시간이 길어집니다.



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **조건부 분기 필수:** 인텐트 분류 결과에 따라 다른 도구나 모델을 동적으로 선택하는 하이브리드 챗봇
- ✓ **복잡한 워크플로우:** 여러 단계의 데이터 처리와 복잡한 비즈니스 로직 제어가 필요한 시스템
- ✓ **인간 개입(HITL):** 중간 단계에서 사용자의 승인이나 추가 정보 입력이 필요한 워크플로우



주의점 (Considerations) 도입 시 고려사항

2개

- ⓘ **이중화 및 스케일링:** 단일 장애점(SPOF) 방지를 위해 오케스트레이터의 이중화 구성과 스케일링을 반드시 고려해야 합니다.
- ⓘ **서킷브레이커 패턴:** 하위 Worker 서비스의 장애가 중앙으로 전파되는 것을 막기 위해 서킷브레이커 적용이 권장됩니다.

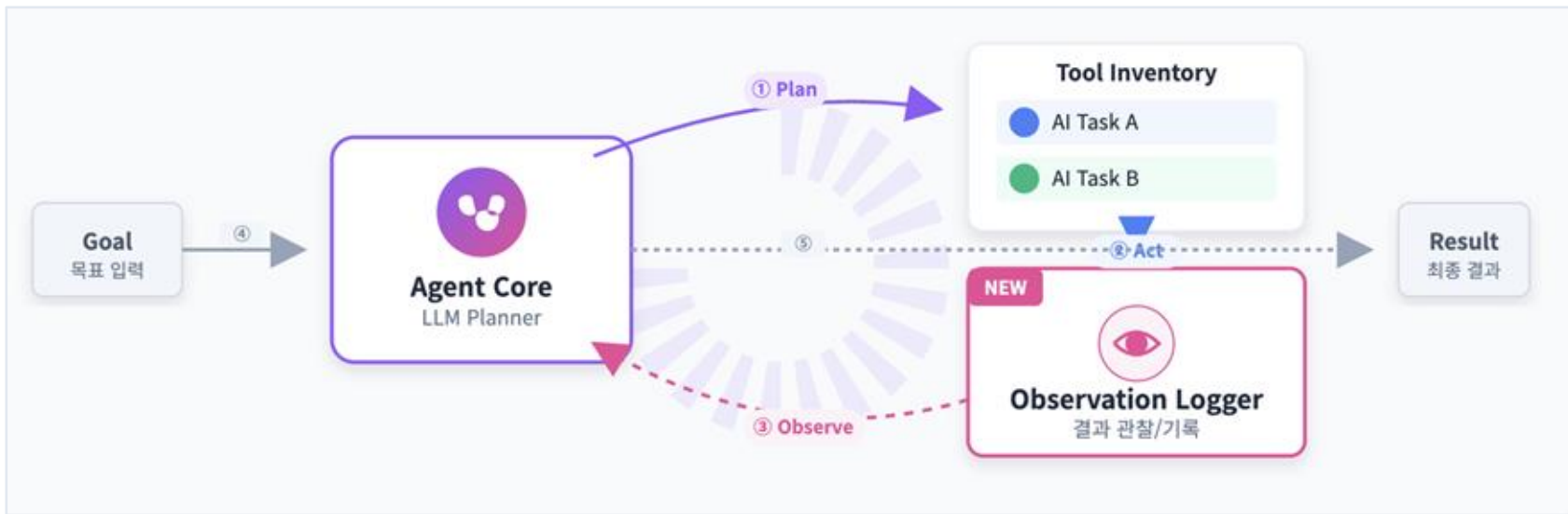


자율적 에이전트 루프 패턴 (Autonomous Agentic Loop Pattern)

목표 달성을 위해 AI가 스스로 실행 경로를 탐색하며, '계획-실행-관찰'의 반복적인 피드백 루프를 통해 비정형적이고 복잡한 문제에 대응하는 전략입니다.



컴포넌트 및 커넥터 구조 Component and Connector View



1 자율적 계획 (Plan)

Agent Core가 목표에서 실행 계획을 수립하고 적절한 도구를 선택합니다.

2 실행 및 관찰 (Act+Observe)

선택된 Tool을 실행한 후, Observation Logger가 실행 결과를 세밀하게 기록합니다.

3 피드백 루프 (Loop)

관찰 결과를 바탕으로 목표 달성 여부를 판단하고, 필요 시 계획을 수정(Replan)하여 반복합니다.



커넥터 정보

Feedback Loop Connector

실행-관찰-재계획의 반복 루프 흐름

Observation Channel

Tool 실행 결과를 세밀하게 추적 및 기록



주요 특징

- 상황에 따른 동적 경로 탐색
- 관찰을 통한 자가 수정(Self-correction)
- 사전 정의하기 힘든 비정형 문제 대응
- 단순 절차가 아닌 목표지향 탐색



자율적 에이전트 루프 패턴 평가

목표 달성을 위해 AI가 스스로 실행 경로를 탐색하고 자가 수정하는 패턴의 실무 적용성과 주의사항을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **비정형 문제 대응:** 사전에 정해지지 않은 문제에 AI가 스스로 도구를 선택하여 실행합니다.
- ✓ **자가 수정 능력:** 실행 결과가 불만족스러울 시 다른 경로를 탐색하여 스스로 재시도합니다.
- ✓ **높은 유연성:** 설계 시점에 실행 순서를 정의할 수 없는 고난도 복잡한 문제에 대응 가능합니다.



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **예측 가능성 저하:** AI의 경로 선택을 통제하기 어려워 무한 루프나 오류 발생 가능성이 있습니다.
- ⚠ **비용 효율성 한계:** 고성능 추론 모델의 반복적 호출로 인해 API 비용과 지연 시간이 증가합니다.
- ⚠ **디버깅 복잡도:** 동적이고 비선형적인 실행 경로로 인해 에러 추적 및 원인 파악이 어렵습니다.



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **목표지향 탐색 과제:** 고정된 워크플로우로 사전에 모든 분기를 정의할 수 없는 비정형 문제
- ✓ **연구 및 개발 보조:** 복잡한 코딩 에이전트, 논문 요약 및 리서치, 다단계 데이터 분석 시스템
- ✓ **동적 의사결정:** 환경의 변화나 이전 행동의 결과에 따라 다음 행동 계획을 수정해야 하는 작업



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **반복/비용 한도 제한:** 무한 루프를 방지하기 위해 최대 반복 횟수(Max Iterations)와 비용 상한 설정 필수
- ⓘ **안전 가드레일:** 외부 시스템 제어 시 톨 사용 권한 제한 및 치명적 행동 사전 방지 승인 메커니즘 필요
- ⓘ **가시성 및 모니터링:** 에이전트의 관찰, 계획, 행동 로그를 기록하여 디버깅 및 분석에 적극 활용

AI 서비스 결정 (AI Service Discovery & Selection)

애플리케이션이 추론을 수행할 때, 여러 개의 가용한 AI 서비스 인스턴스 중 '어떤 서비스(모델)를 호출할 것인가'를 런타임에 결정하는 메커니즘을 다룹니다. 특정 모델 엔드포인트에 종속되지 않으면서도, 상황(비용, 성능, 가용성 등)에 따라 최적의 서비스를 선택할 수 있는 유연한 구조를 설계해야 합니다.

3

하위 패턴

2-1. 정적 엔드포인트 바인딩

Static Endpoint Binding

호출 대상을 코드/설정 수준에서 특정 엔드포인트로 고정

탐색이나 중재 과정 없이 즉시 연결 수행

통신 오버헤드 제거, 구조적 단순성 극대화

단순성

● 최저지연

2-2. 클라이언트 사이드 디스커버리

Client-side Discovery

클라이언트가 직접 가용한 서비스 목록을 조회하고 선택

레지스트리로부터 가용 서비스 정보를 획득

클라이언트 수준의 로드밸런싱으로 호출 대상 결정

가용성

● 분산처리

2-3. 서비스 브로커/라우터

Service Broker/Router

애플리케이션과 AI 제공자 사이에 중재자 배치

정책(비용, 품질 등)에 따라 모델 동적 바인딩

멀티 모델 운영의 유연성 확보

유연성

● 동적라우팅

설계 초점

지연 vs 유연성

장애 내성

변경 용이성

비용 최적화



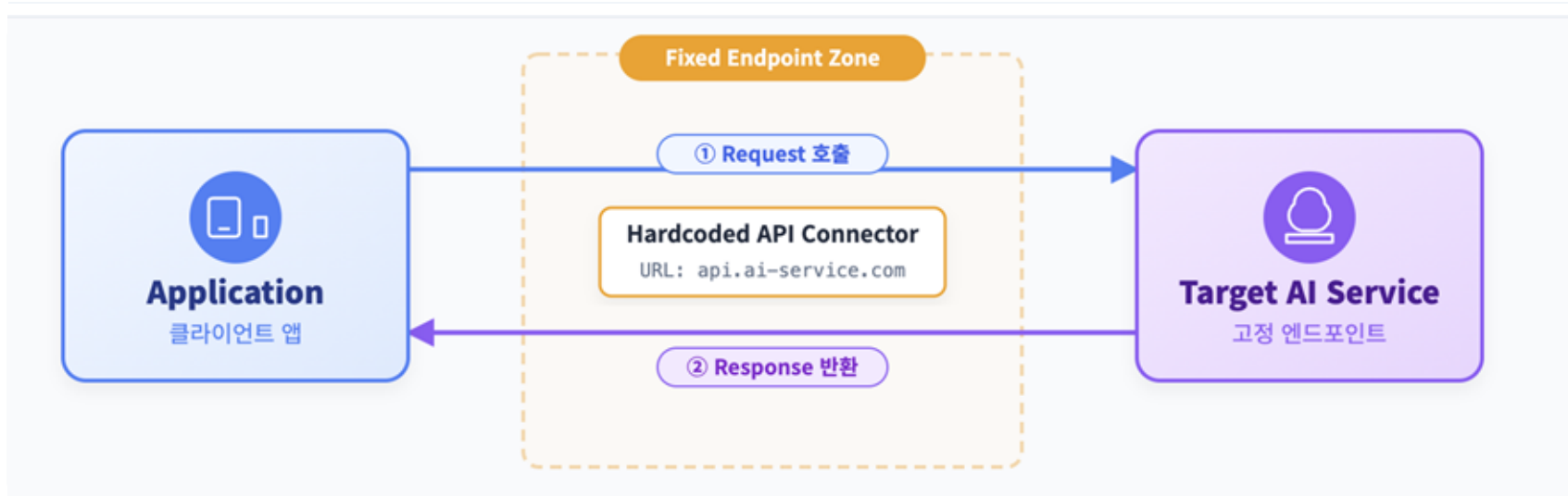
정적 엔드포인트 바인딩 패턴 (Static Endpoint Binding Pattern)

호출 대상 엔드포인트를 코드나 설정에 고정하여 중재 없이 즉시 연결함으로써 통신 오버헤드를 제거하는 전략입니다.



컴포넌트 및 커넥터 구조

Component and Connector View



1 하드코딩 API 커넥터

고정된 URL/IP 주소를 가진 커넥터로, 탐색이나 중재 과정 없이 즉시 목표 AI 서비스를 호출합니다.

2 1:1 직접 연결

Application이 Target AI와 직접 통신하며, 중간에 프록시나 라우터 같은 별도의 레이어가 없습니다.

3 오버헤드 최소화

별도의 탐색이나 중재 과정 없이 즉시 호출이 발생하므로 네트워크 지연 시간 (Latency)이 최소화됩니다.



커넥터 정보

Hardcoded API Connector

고정된 URL/IP 주소를 코드나 설정에 박아둔 커넥터

Direct Connection (Req)

Application → Target AI Service

Direct Connection (Res)

Target AI Service → Application



주요 특징

- ✓ 구조 단순성: 중재자가 없어 설계와 구현이 매우 단순함
- ✓ 오버헤드 無: 네트워크 홉(hop) 추가가 없어 가장 빠른 응답 속도
- ✓ 예측 가능성: 항상 동일한 모델/엔드포인트가 호출됨
- ⚠ 변경 어려움: 엔드포인트 변경 시 코드 수정 및 앱 재배포 필수



2-1 정적 엔드포인트 바인딩 - 평가/적용

장점·단점·적용·주의점 분석



정적 엔드포인트 바인딩 패턴 평가

호출 대상을 코드/설정에 고정하여 구조를 단순화하고 중재 오버헤드를 제거한 패턴의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **예측 가능성:** 항상 같은 AI 서비스가 호출되므로, 시스템 동작을 이해하기 쉽다
- ✓ **구조 단순성:** 별도 인프라 없는 직관적 1:1 연결로 복잡도 최소화
- ✓ **빠른 응답:** 중개 노드가 없어 네트워크 지연(Latency) 최소화



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **변경 시 재배포:** 엔드포인트 변경 시 애플리케이션 코드 수정 및 재배포 필수
- ⚠ **장애 대체 불가:** 특정 대상 서비스 장애 발생 시 즉각적 우회 처리 불가
- ⚠ **유연성 부족:** 런타임 동적 모델 선택이나 트래픽 로드밸런싱 미지원



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **온디바이스 단일 모델:** 모바일/IoT 기기에서 고정된 내장 AI 모델 호출 시
- ✓ **변화 적은 서비스:** 타겟 AI 엔드포인트 주소가 장기간 고정 유지되는 환경
- ✓ **성능 최우선:** 검색 지연조차 허용되지 않는 초저지연 필수 비즈니스 로직



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **설정 외부화:** 하드코딩 피하고 환경 변수/설정 파일로 엔드포인트 분리
- ⓘ **장애 시 폴백:** 클라이언트 단에서 수동 대체 엔드포인트 우회 로직 구현
- ⓘ **모니터링 알람:** 대상 엔드포인트 가용성 및 헬스 체크 상태 지속 모니터링



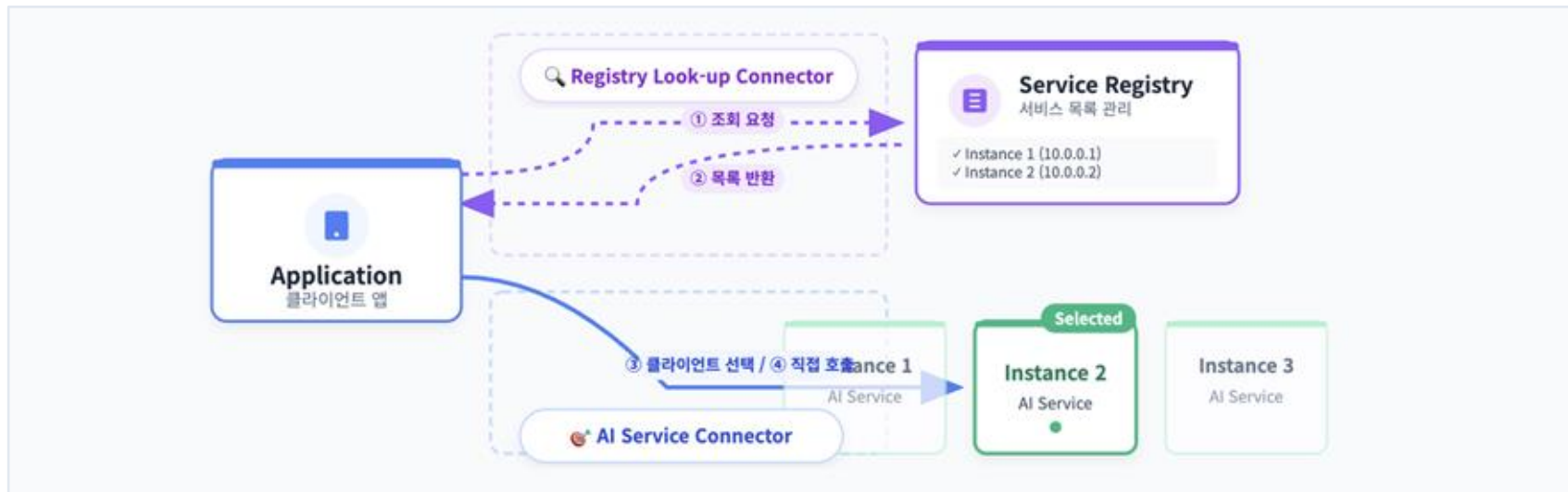
클라이언트 사이드 디스커버리 패턴 (Client-side Discovery Pattern)

클라이언트가 레지스트리로부터 가용 서비스 목록을 조회한 후 직접 선택하여 수행하는 전략입니다.



컴포넌트 및 커넥터 구조

Component and Connector View



1 레지스트리 조회

Application이 Service Registry에 가용 서비스 목록을 요청합니다.

2 클라이언트 선택

반환된 목록 중 인스턴스를 결정합니다.

3 직접 통신

선택된 AI Service Instance와 직접 호출하여 추론을 수행합니다.



커넥터 정보

Registry Look-up Connector

가용 서비스 목록 조회용 커넥터

AI Service Connector

선택된 인스턴스와의 직접 통신



주요 특징

- 중앙 병목 없이 부하 분산 가능
- 클라이언트에 선택 로직 구현이 필요해 복잡도 증가
- 레지스트리 서비스에 대한 강한 의존성



클라이언트 사이드 디스커버리 패턴 평가

클라이언트가 레지스트리에서 서비스 목록을 조회하여 직접 선택하는 전략의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **가용성 향상:** 장애 발생 시 레지스트리에서 정상 인스턴스를 찾아 호출합니다.
- ✓ **부하 분산:** 클라이언트가 로드밸런싱 알고리즘으로 효율적인 트래픽 분산 수행
- ✓ **동적 탐색:** 런타임에 가용한 서비스 인스턴스를 동적으로 탐색하여 유연성 확보



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **클라이언트 복잡도 증가:** 서비스 목록 관리 및 레지스트리 통신 로직 추가 필요
- ⚠ **레지스트리 의존성:** 서비스 레지스트리 가용성에 종속되어 장애 시 영향 발생
- ⚠ **네트워크 오버헤드:** 서비스 조회를 위한 추가적인 네트워크 호출 비용 발생



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **마이크로서비스 아키텍처:** 다수의 모델 인스턴스가 동적으로 증설/감축되는 환경
- ✓ **분산 추론 환경:** 클라이언트가 직접 최적의 인스턴스를 선택해야 하는 상황
- ✓ **고가용성 요구:** 특정 노드 장애 시 즉각적으로 다른 인스턴스로 자동 전환 필요시



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **헬스체크/TTL:** 인스턴스 건강 상태 주기적 확인 및 TTL 설정 필수 적용
- ⓘ **캐시 풀백:** 레지스트리 장애 대비 로컬 캐시를 활용한 풀백 전략 구현 필수



서비스 브로커/라우터 패턴 (Dynamic Service Broker/Router Pattern)

애플리케이션과 AI 제공자 사이의 중재자가 정책(비용, 성능)에 따라 동적으로 최적의 모델을 선택하는 구조입니다.



컴포넌트 및 커넥터 구조

Component and Connector View



1 정책 기반 라우팅

비용, 지연 시간, 품질 등의 정의된 정책에 따라 런타임에 최적의 AI Provider를 동적으로 선택합니다.

2 호출 분리

애플리케이션은 논리적 서비스명으로만 요청하며, 브로커가 실제 물리적 엔드포인트를 결정하여 결합도를 낮춥니다.

3 중재 계층 역할

단순한 라우팅을 넘어 요청/응답을 중계하며 중앙 집중형 로깅, 모니터링, 오류 시 폴백(Fallback) 처리를 수행합니다.



커넥터 정보

Policy-based Router

앱의 논리적 요청을 받아 브로커의 라우팅 정책 엔진으로 전달하는 단일 진입점

Service Connector

평가 결과에 따라 동적으로 선택된 실제 AI Provider 인스턴스와의 통신 채널



주요 특징

- **유연성 확보:** 런타임에 모델 변경 및 로드 밸런싱이 자유로움
- **모델 교체 용이:** 클라이언트 코드 수정 없이 새로운 AI 모델 추가 가능
- **중앙 집중 관리:** API 키, 쿼터 제한, 로깅 등을 통합 관리
- **고가용성:** 특정 모델 장애 시 다른 모델로 즉시 폴백(Fallback) 가능



서비스 브로커/라우터 패턴 평가

중재자가 정책(비용, 성능)에 따라 모델을 동적 선택하는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages)

- ✓ **모델 교체 용이:** 앱 수정 없이 브로커에서 모델 교체 가능
- ✓ **멀티모델 유연성:** 여러 AI 모델을 동시에 운영하며 상황에 따라 최적의 모델을 선택할 수 있습니다.
- ✓ **정책 기반 라우팅:** 비용, 성능, 가용성 등 다양한 정책에 따라 모델을 동적으로 선택할 수 있습니다.



단점 (Disadvantages)

- ⚠ **추가 홉 지연:** 모든 요청이 브로커라는 추가 홉을 거치므로 지연 시간이 발생합니다.
- ⚠ **운영 복잡도:** 고가용성 브로커 인프라 구축 및 관리 필요
- ⚠ **단일 장애점:** 브로커 장애 시 전체 시스템이 영향을 받을 수 있습니다.



적합한 상황 (Use Cases)

- ✓ **멀티모델 운영:** 여러 AI 모델을 동시에 사용해야 하는 환경
- ✓ **비용/성능 정책:** 비용, 성능 등 정책에 따라 모델을 선택해야 하는 경우
- ✓ **무중단 교체:** 서비스 중단 없이 모델 전환 필요 시



주의점 (Considerations)

- ⓘ **고가용성 구성:** 브로커의 이중화와 장애 복구 메커니즘을 설계해야 합니다.
- ⓘ **SLO 관리:** 지연 시간 SLO를 설정하고 모니터링하여 성능을 보장해야 합니다.
- ⓘ **라우팅 로그/관측성:** 라우팅 결정을 로깅하고 모니터링해야 합니다.



AI 서비스 배포 (AI Service Deployment)

AI 모델의 물리적 실행 위치와 인프라 구조를 결정하는 문제. 지연시간·보안·자원 비용 사이의 균형점을 찾아 최적 환경을 배치합니다.

3

하위 패턴

디바이스 (Edge)

클라우드 (Cloud)

균형 (Hybrid)



3-1. 로컬 엣지 배포

Local Edge Deployment

배포 위치 Edge 100%

- ✓ 단말 내에서 모든 추론을 처리하여 네트워크 의존성 제거
- ✓ 데이터 외부 유출 방지하여 보안성 확보
- ✓ 오프라인 상태에서도 AI 서비스 이용 가능

저지연

● 프라이버시



3-2. 중앙 서버 배포

Centralized Server

배포 위치 Cloud 100%

- ✓ 고성능 GPU 자원 활용하여 복잡한 모델 서빙
- ✓ 모델 업데이트와 관리가 용이
- ✓ 트래픽 증가 시 스케일아웃 가능

확장성

● 중앙관리



3-3. 하이브리드 분할 배포

Hybrid Split-Deployment

배포 위치 Hybrid 50:50

- ✓ 로컬 전처리 + 서버 정밀 분석으로 부하 분산
- ✓ 성능과 비용의 균형 맞춤
- ✓ 네트워크 장애 시 로컬 폴백 가능

균형

● 연속성



설계 초점



지연 시간 최소화



보안/프라이버시



자원/비용 효율성



네트워크 연속성



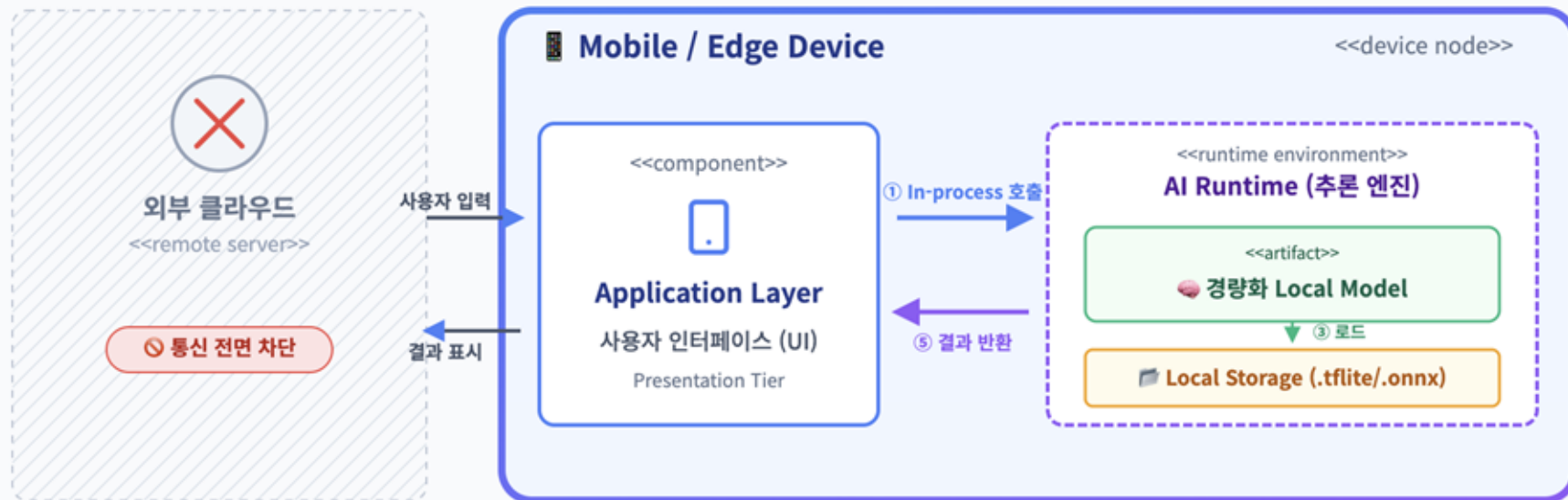
로컬 엣지 배포 패턴 (Local Edge Deployment Pattern)

데이터를 기기 내부에서 처리하여 네트워크 의존성을 제거하고 외부 유출을 원천 방지하는 배포 전략입니다.



디바이스 노드 및 컴포넌트 배포 (Deployment View)

물리적 장치 내부의 소프트웨어 아티팩트 및 런타임 환경



1 사용자 입력

사용자가 Application(UI)을 통해 데이터 입력

2 추론 요청

앱이 In-process 호출로 AI Runtime 실행

3 모델 로딩

런타임이 스토리지에서 경량화 모델 로드

4 로컬 추론

디바이스 자원으로 추론 (네트워크 통신 無)

5 결과 반환

추론 결과를 앱으로 반환하여 화면 표시

커넥터 정보

In-process Call

앱과 런타임 간의 동일 프로세스 내 직접 함수 호출 (오버헤드 최소화)

Local IPC

디바이스 내 독립 프로세스 간 데이터 통신 (네트워크 미경유)

핵심 가치

초저지연 보장

네트워크 RTT가 0에 수렴하여 실시간 응답 가능

완벽한 프라이버시

민감 데이터가 디바이스 외부로 유출되지 않음

완전한 오프라인 동작

네트워크 단절 상태에서도 AI 서비스 지속 제공

디바이스 자원 활용

엣지 기기의 NPU/GPU를 직접 활용하여 연산



로컬 엣지 배포 패턴 평가

기기 내부에서 추론 처리하여 네트워크 의존성을 제거하고 데이터 유출을 방지하는 패턴의 실무 적용성을 분석합니다



장점 (Advantages)

- ✓ **초저지연:** 네트워크 지연이 없어 즉각적인 추론 결과 반환 (Real-time)이 가능합니다.
- ✓ **프라이버시 보호:** 민감한 데이터를 외부 서버로 전송하지 않고 로컬 기기 내에서 처리합니다.
- ✓ **오프라인 동작:** 인터넷 연결 없이도 AI 서비스 이용이 가능합니다.



단점 (Disadvantages)

- ⚠ **연산/메모리 한계:** 제한된 자원으로 대형 모델 실행 어려움
- ⚠ **업데이트 어려움:** 다수 기기의 모델 일관 업데이트 곤란
- ⚠ **모델 크기 제한:** 경량화된 모델만 사용 가능하여 정확도가 낮을 수 있습니다.



적합한 상황 (Use Cases)

- ✓ **IoT/산업 현장:** 실시간 센서 데이터 처리가 필수적인 환경
- ✓ **민감 데이터 환경:** 의료·금융 등 개인정보 유출 차단 필수 환경
- ✓ **오프라인 필수:** 네트워크 연결이 불안정하거나 없는 환경



주의점 (Considerations)

- ⓘ **모델 경량화/양자화:** 엣지 환경에 맞는 모델 최적화가 필요합니다.
- ⓘ **보안 저장:** 모델 파일의 안전한 저장 및 접근 권한 관리가 필요합니다.
- ⓘ **디바이스 매트릭스 테스트:** 다양한 디바이스에서의 성능 테스트가 필요합니다.



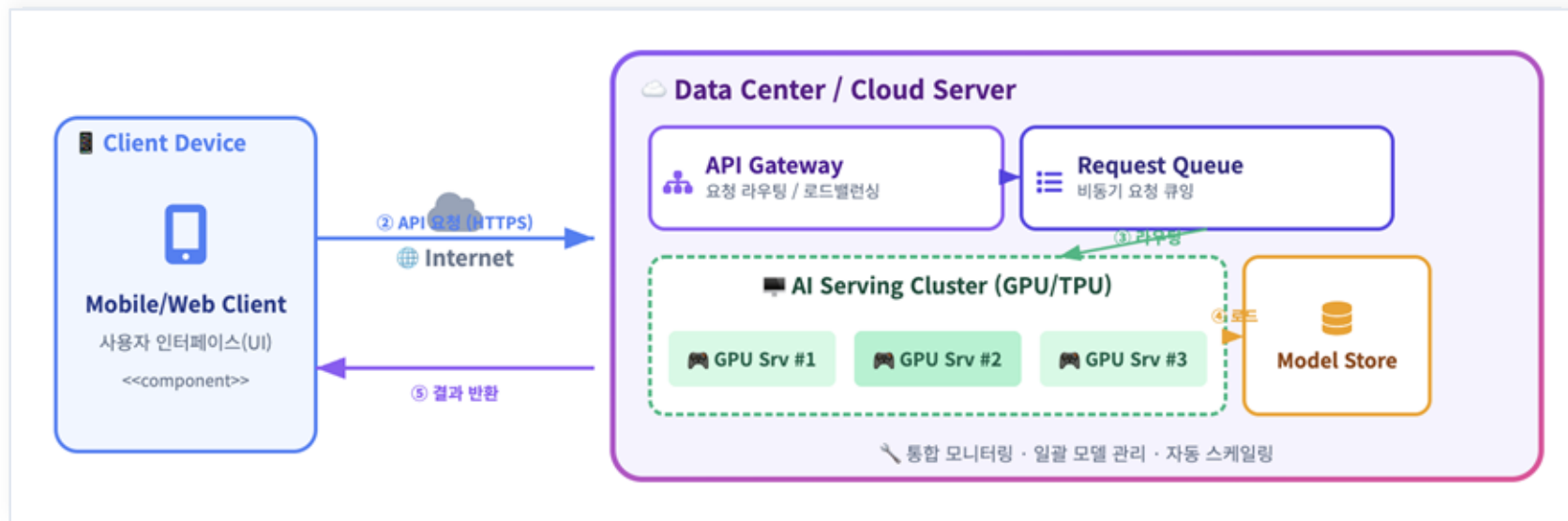
중앙 서버 배포 패턴 (Centralized Server Deployment)

클라이언트는 입출력만 수행하고 복잡한 추론은 고성능 중앙 서버에 위임하여 자원을 효율적으로 활용하는 배포 전략입니다.



서버 노드 및 컴포넌트 배포 (Deployment View)

고성능 클라우드 인프라의 추론 서빙 환경



1 사용자 입력

클라이언트 앱에서 사용자가 데이터 입력

2 API 요청

HTTPS Secure Channel로 추론 요청 전송

3 큐잉 & 라우팅

Gateway가 Queue를 거쳐 GPU 서버로 분배

4 GPU 추론

고성능 서버가 모델 로드 후 추론 수행

5 결과 반환

네트워크를 통해 추론 결과를 앱으로 응답



커넥터 정보



HTTPS / Network API

클라이언트와 서버 간 표준 네트워크 통신



Secure Channel (TLS)

암호화된 보안 채널로 전송 데이터 보호



Load Balancer

다중 GPU 인스턴스 간 원활한 트래픽 분산



핵심 가치



고성능 연산: GPU/TPU 클러스터로 대형 모델 서빙 가능



일괄 관리: 중앙에서 모델 버전 및 배포 통합 관리



확장성 (Scale-out): 트래픽 증가 시 유연한 인스턴스 추가



3-2 중앙 서버 배포 - 평가/적용

장점·단점·적용·주의점 분석



중앙 서버 배포 패턴 평가

고성능 GPU/TPU로 대형 모델을 서빙하며, 업데이트가 용이한 중앙 집중형 구조의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **고성능 연산 능력:** GPU/TPU로 복잡한 대형 모델을 효율적으로 실행합니다.
- ✓ **업데이트 용이:** 중앙 서버의 모델만 교체하면 전체 서비스에 즉시 반영됩니다.
- ✓ **스케일아웃:** 트래픽 증가 시 서버 인스턴스를 동적으로 늘려 유연한 대응이 가능합니다.



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **네트워크 지연:** 네트워크 환경에 종속되어 실시간성이 필요한 환경에서는 응답 지연이 발생할 수 있습니다.
- ⚠ **네트워크 의존성:** 서버 장애나 네트워크 단절 시 AI 서비스를 전혀 이용할 수 없는 위험이 있습니다.
- ⚠ **운영 비용:** 고성능 GPU/TPU 등 서버 인프라 유지에 높은 운영 비용이 지속적으로 발생합니다.



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **대규모 웹 서비스:** 다수의 사용자에게 중앙화된 AI 서비스를 안정적으로 제공해야 하는 환경에 적합합니다.
- ✓ **거대 LLM 서빙:** 디바이스 탑재가 불가능한 초거대 언어 모델(LLM)을 서비스하는 경우 필수적입니다.
- ✓ **빈번한 업데이트:** 지속적인 모델 학습과 배포가 이루어져 클라이언트 업데이트가 부담스러운 경우 유리합니다.



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **오토스케일:** 예측 불가능한 트래픽 변동에 대비하여 자동 인스턴스 조정 정책을 세밀하게 설계해야 합니다.
- ⓘ **암호화:** 네트워크를 통한 데이터 전송 시 민감 정보 유출 방지를 위한 엔드투엔드 암호화가 필수적입니다.
- ⓘ **지역 최적화:** 글로벌 서비스의 경우 사용자 위치에 따른 지연을 최소화하기 위해 엣지 로케이션을 고려해야 합니다.



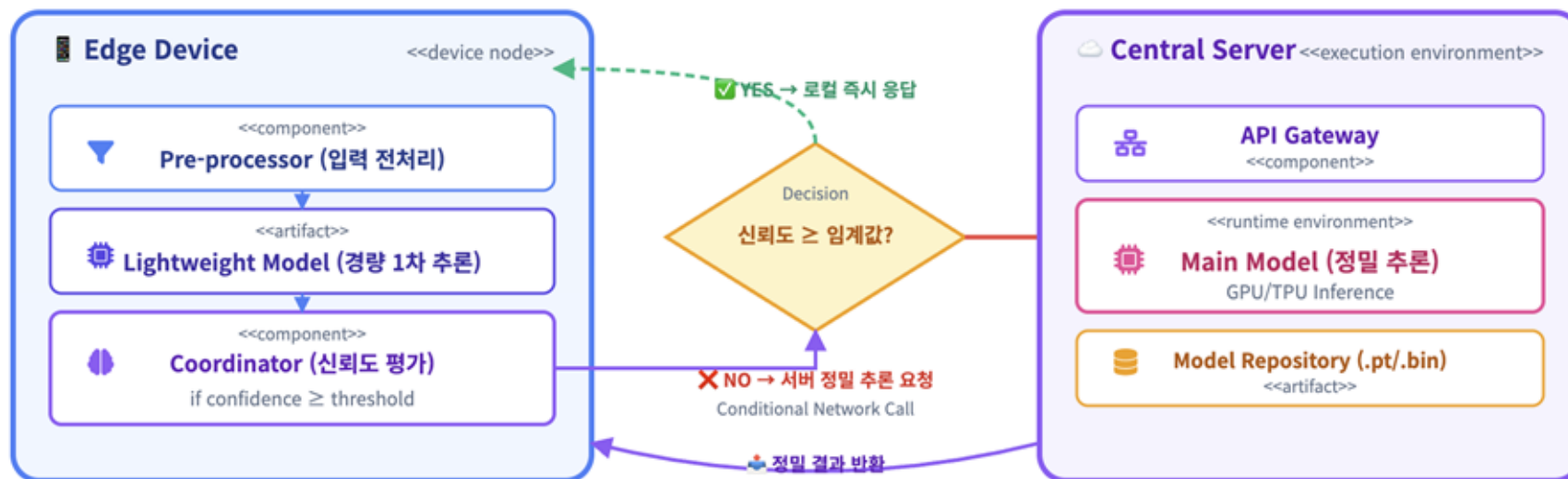
하이브리드 분할 배포 패턴 (Hybrid Split-Deployment Pattern)

추론 단계를 논리적으로 분할하여 로컬 기기와 원격 서버가 역할을 분담하는 협업 전략입니다.



엣지-서버 분할 배포 (Deployment View)

신뢰도 기반 조건부 분기 협업 구조



1 사용자 입력

Edge Device의 Pre-processor가 입력 데이터 수집·전처리

2 1차 추론

Lightweight Model이 디바이스에서 경량 추론 수행

3 신뢰도 평가

Coordinator가 결과 신뢰도를 임계값과 비교

4 조건부 분기

신뢰도 충분 → 로컬 응답
부족 → Server 호출

5 결과 반환

로컬 또는 서버에서 최종 결과를 사용자에게 전달

커넥터 정보

Local IPC / API

디바이스 내부 컴포넌트 간 통신 (저지연)

Conditional Network Call

신뢰도 기반 조건부 서버 호출

HTTPS / Secure Channel

Edge ↔ Server 간 보안 통신

핵심 가치

성능/비용 균형

로컬과 클라우드 자원의 최적 분담

장애 시 폴백

서버 장애 시 로컬 응답으로 대체 가능

데이터 필터링

민감 데이터는 로컬 처리, 일부만 전송

신뢰도 기반 라우팅

임계값 정책으로 트래픽 동적 분배



하이브리드 분할 배포 패턴 평가

로컬 엣지와 중앙 서버의 역할 분담으로 성능과 비용의 균형을 맞추는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages)

- ✓ **성능/비용 균형:** 엣지 전처리-서버 고도추론 분할로 비용 절감
- ✓ **로컬 풀백 가능:** 네트워크 장애 시 로컬 모델로 즉시 전환
- ✓ **통신비 절감:** 모든 데이터를 클라우드로 전송하지 않아 비용 절감



단점 (Disadvantages)

- ⚠ **구조 복잡:** 엣지와 서버 양쪽의 런타임 환경 설계가 필요합니다.
- ⚠ **데이터 동기화:** 로컬-서버 간 모델 버전 및 데이터 동기화 복잡
- ⚠ **운영 복잡성:** 두 환경을 모두 모니터링하고 관리해야 합니다.



적합한 상황 (Use Cases)

- ✓ **통신비 절감:** 대량 데이터 처리 시 통신비용 최소화
- ✓ **연속성 보장 필요:** 네트워크 장애 시에도 서비스를 유지해야 함
- ✓ **혼합 워크로드:** 실시간 처리와 고성능 추론이 모두 필요한 경우



주의점 (Considerations)

- ⓘ **임계값 튜닝:** 로컬/서버 전환 기준을 적절히 설정해야 합니다.
- ⓘ **버전 정렬:** 엣지와 서버의 모델 버전을 일치시켜야 합니다.
- ⓘ **프라이버시 규정:** 민감한 정보는 로컬에서만 처리해야 합니다.



컨텍스트 획득 정확성 (Context Acquisition Accuracy)

멀티턴 대화나 시계열 태스크에서 모델 외부의 상태(State)와 기억(Memory)을 어떻게 누락이나 왜곡 없이 무결하게 유지할 것인가에 대한 문제입니다.

세션 정보를 정확하게 보존하여 시가 이전 맥락을 완벽하게 이해하고 활용할 수 있도록 하는 것이 핵심 목표입니다.

3

하위 패턴

원시 데이터 보존 (Raw Data)

정보 압축 (Compression)

구조화 및 검색 (Structured)



4-1. 전체 이력 버퍼

Full History Buffer

정확성 (High)

토큰 효율성 (Low)

- ✓ 과거의 모든 데이터 시퀀스를 가공 없이 원본 상태로 보존
- ✓ 정보의 가공이나 누락 없이 100% 맥락 참조 가능
- ✓ 순차적 메시지 저장 및 Raw Context Provider 활용

무결성

● 정확성 최고



4-2. 압축 요약 기억

Summarized Memory

정확성 (Med)

토큰 효율성 (High)

- ✓ 누적되는 이력을 추상화된 형태의 요약문으로 압축
- ✓ 토큰 소비를 최소화하면서 핵심 맥락 유지
- ✓ Context Analyzer와 Memory Summarizer 활용

효율성

● 토큰 절감



4-3. 세션 계층화 검색

Hierarchical Retrieval

정확성 (High)

토큰 효율성 (High)

- ✓ 단기 캐시와 장기 벡터 스토어로 계층화
- ✓ 필요한 정보를 동적으로 검색하여 정확성 확보
- ✓ Semantic Retriever로 연관 정보 정확 회상

확장성

● 대규모 지원



설계 초점



정확성 vs 토큰 비용



확장성 고려



검색 신뢰성



실시간 처리



전체 이력 버퍼 패턴 (Full History Buffer Pattern)

모든 데이터 시퀀스를 원본 상태로 보존하여 맥락의 100% 무결성을 보장하는 패턴의 내부 모듈 구성입니다.



모듈 분해 구조 (Module View)

3대 핵심 컴포넌트 기반 아키텍처

Full History Buffer Modules



모듈 간 인터페이스

Session Context API

세션 ID 기반 메시지 식별 및 저장소 라우팅 연동

Message Stream API

시간순으로 누적된 원본 메시지 시퀀스 무결성 제공

Full Context Payload

가공 없는 전체 이력을 모델의 입력 형식으로 직렬화

✓ 핵심 아키텍처 특성

100% 맥락 무결성

요약이나 필터링 과정 생략으로 정보 손실 제로

순차적 접근 구조

시간 흐름 기반의 가장 단순하고 명확한 관리

토큰 비용 종속성

입력 사이즈 선형 증가 구조

1 입력 수신 및 식별

Session Manager가 사용자 입력을 수신하고 고유 세션 ID 매핑

2 원본 이력 보존

Sequential Message Store에 모든 데이터 시퀀스를 가공 없이 보존

3 100% 맥락 참조

Raw Context Provider가 정보의 가공 없이 전체 이력 데이터 조회

4 원본 컨텍스트 제공

직렬화된 메시지 전체를 최종 AI 모델의 입력으로 100% 전달



전체 이력 버퍼 패턴 평가

모든 데이터 시퀀스를 원본 상태로 보존하여 맥락의 100% 무결성을 보장하는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **기능 정확성 최고:** 과거 모든 맥락을 100% 참조 가능
- ✓ **구현이 단순:** 복잡한 검색 시스템이나 요약 로직이 필요 없습니다.
- ✓ **신뢰성:** 요약 과정에서 발생할 수 있는 왜곡 위험이 없습니다



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **토큰 비용 급증:** 이력이 길어질수록 모델 입력(Token) 비용이 급증합니다
- ⚠ **컨텍스트 한계:** 모델 컨텍스트 길이 초과 위험
- ⚠ **성능 저하:** 대량의 데이터 처리로 인해 응답 속도가 느려질 수 있습니다



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **짧은 세션:** 대화 턴 수가 비교적 짧은 환경
- ✓ **법률/의료:** 정확성이 절대적으로 중요한 영역
- ✓ **감사/기록:** 모든 대화 내용을 완벽하게 보존해야 하는 경우



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **토큰 한계 모니터링:** 최대 토큰 한계 초과 방지
- ⓘ **길이 초과 시 전략:** 길이가 초과될 경우 전략적 절단이 필요합니다
- ⓘ **비용 관리:** 토큰 비용을 고려한 비용 관리가 필요합니다



압축 요약 기억 패턴 (Summarized Memory Pattern)

누적된 이력이 임계치를 초과할 때 추상화된 요약문으로 압축하여 맥락을 유지하면서 토큰 소비를 최소화하는 전략입니다.



동적 압축 및 상태 보존 메커니즘

Many-to-One Compression Mechanism



1 임계치 자동 감지

Context Analyzer가 누적 토큰을 모니터링하여 임계치 초과 시 요약 프로세스를 자동으로 트리거합니다.

2 의미 중심 압축

Memory Summarizer가 이전 대화의 핵심 맥락을 추출하여 단일 요약문으로 추상화(Many→One) 합니다.

3 효율적 상태 보존

이후 추론 시 Store에 보관된 요약본과 최신 입력 메시지를 조합하여 모델에 전달, 장기 대화를 지원합니다.

i 커넥터 정보

Summary Trigger

토큰/메시지 수 기반 요약 트리거 로직

Update Pipeline

기존 요약 + 새 이력 → 신규 요약 업데이트

✓ 핵심 효과 및 특징

- **토큰 절감 효과**
입력 데이터 증가에도 고정된 비용 유지
- **응답 속도 향상**
모델의 Context 처리량 감소로 추론 속도 개선
- **장기 대화 유지**
토큰 한계로 인한 세션 종료 방지 (무한 세션)
- **정보 왜곡 위험**
압축 과정에서 세부 데이터 손실 가능성 존재



4-2 압축 요약 기억 - 평가/적용

장점·단점·적용·주의점 분석



압축 요약 기억 패턴 평가

누적 이력을 추상화된 요약문으로 압축하여 토큰 제한 문제를 해결하는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **토큰 절감:** 방대한 이력을 요약문으로 압축하여 토큰 소비 최소화
- ✓ **속도 향상:** 요약된 맥락으로 인해 추론 속도가 빨라집니다
- ✓ **장기 대화 유지:** 무한히 길어지는 대화 흐름을 유지하기에 유리합니다



단점 (Disadvantages) 패턴의 본질적 한계

3개

- ⚠ **세부정보 손실:** 구체적 수치, 고유명사, 미세 맥락 소실 위험
- ⚠ **보조 모델 필요:** 주기적인 요약을 위한 추가 모델/프롬프트 실행 비용 증가
- ⚠ **요약 편향:** 중요 정보가 누락되거나 의도와 다르게 왜곡될 가능성



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **일상 대화형 비서:** 정확한 세부 정보보다는 전반적인 긴 대화 맥락 유지가 중요한 환경
- ✓ **롤플레이팅 게임/캐릭터 AI:** 페르소나 설정과 이전 스토리의 큰 흐름 유지가 필수적인 서비스
- ✓ **온디바이스 AI:** 메모리와 처리 능력이 제한된 로컬 디바이스 환경



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **앵커 팩트 고정:** 이름, 주소 등 절대 손실되어서는 안 될 중요 사실 정보는 별도 관리 필수
- ⓘ **주기적 재요약 정책:** 어느 시점(토큰 수, 턴 수)에 요약을 트리거할지 정교한 정책 필요
- ⓘ **요약 편향 검증:** 생성된 요약문이 원본의 의도를 정확히 반영하는지 모니터링 체계 구축



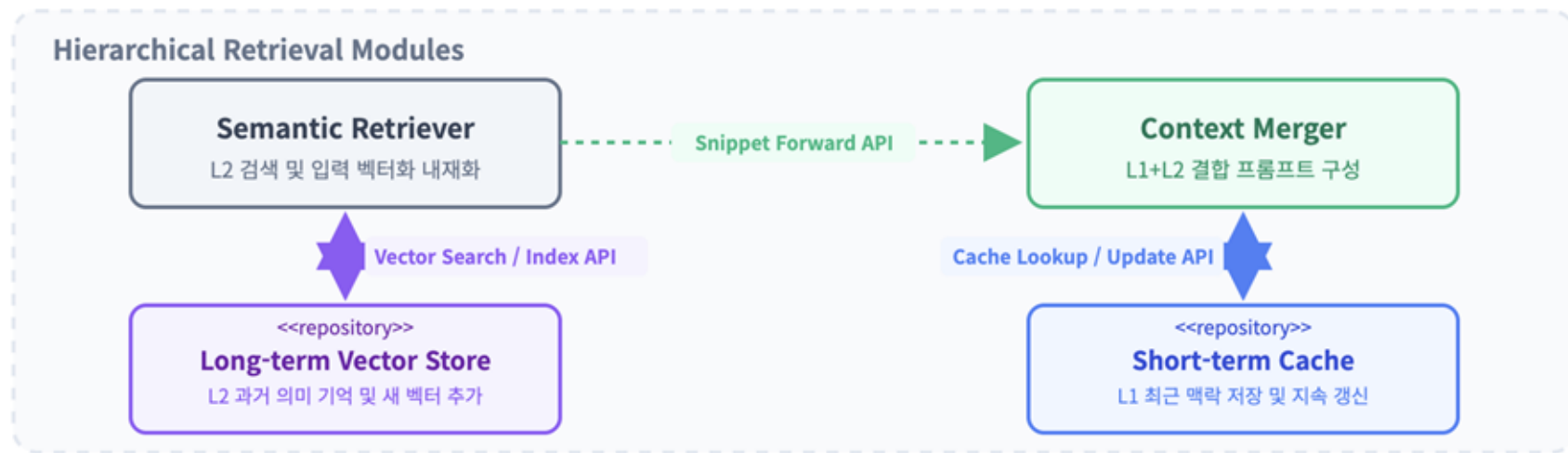
세션 계층화 검색 패턴 (Hierarchical Retrieval Pattern)

단기 캐시와 장기 벡터 스토어를 분리하여 필요한 정보만 동적으로 검색하고 병합하며, 추론 후 피드백 루프를 통해 지속적으로 기억을 갱신하는 아키텍처입니다.



모듈 분해 구조 (Module View)

4대 핵심 컴포넌트 기반 아키텍처 및 피드백 루프



1 새 입력 수신 및 벡터화

Semantic Retriever가 입력을 받아 내부 모델로 쿼리를 벡터화

2 L2 과거 조각 검색

Long-term Vector Store에서 연관된 과거 정보를 검색해 추출

3 L1 최근 맥락 조회

Context Merger가 Short-term Cache에서 최근 맥락을 직접 호출

4 최적 프롬프트 추론

과거 조각과 최근 맥락을 병합하여 AI 모델에 전달 및 추론 수행

5 L1/L2 피드백 갱신

추론 결과를 L1 캐시에 갱신하고 L2 스토어에 새로운 벡터로 추가

모듈 간 인터페이스 (양방향 피드백)

Cache Lookup / Update API

Context Merger가 단기 캐시에서 최근 맥락을 조회하고, 추론 완료 후 새로운 문맥으로 갱신

Vector Search / Index API

내장된 임베딩 모델을 이용해 과거 기억을 검색하며, 처리 완료된 새 데이터 벡터를 장기 저장소에 인덱싱

Snippet Forward API

Semantic Retriever가 검색한 과거 정보 조각을 병합기로 안전하게 전달

✓ 핵심 아키텍처 특성

- 지속적인 상태 갱신 루프
추론 사이클마다 캐시와 장기 저장소가 갱신되어 최신 문맥을 능동적으로 유지
- 무한한 확장성 지원
토큰 한계에 구애받지 않고 방대한 장기 벡터 스토어 활용
- 구조 복잡성 과제
읽기/쓰기 양방향 동기화 및 벡터화 파이프라인으로 인한 시스템 구현 난이도 증가



세션 계층화 검색 패턴 평가

단기 캐시와 장기 벡터 스토어를 분리하여 필요한 정보만 동적 검색하는 패턴의 실무 적용성을 분석합니다.



장점 (Advantages) 패턴이 제공하는 가치

3개

- ✓ **오래된 정보 회상:** 오래전 특정 정보도 검색으로 정확히 찾아 삽입
- ✓ **확장성:** 데이터 증가해도 성능 유지, 대규모 시스템 적합
- ✓ **토큰 효율성:** 필요한 정보만 검색하여 토큰 비용 최소화



단점 (Disadvantages) 본질적 한계

3개

- ⚠ **검색 실패 시 오류:** Retriever가 관련 없는 정보 추출 시 부정확한 답변
- ⚠ **운영 복잡성:** 임베딩/벡터 스토어 관리 및 검색 정확도 튜닝 필요
- ⚠ **구현 난이도:** RAG 시스템 구축 및 유지보수에 전문 지식 필요



적합한 상황 (Use Cases) 실무 적용 시나리오

3개

- ✓ **대규모 지식 축적:** 맞춤형 학습 멘토, BI 에이전트
- ✓ **장기 기억 필요:** 수개월 전 정보가 중요한 시나리오
- ✓ **맥락 기반 검색:** 특정 주제나 문서에서 관련 정보 찾기



주의점 (Considerations) 도입 시 고려사항

3개

- ⓘ **칭킹/임베딩 전략:** 문서 분할 방식과 임베딩 모델의 직접 영향
- ⓘ **RAG 평가/재학습:** 검색 결과의 정확성을 지속적으로 평가
- ⓘ **부적절 문서 필터:** 검색된 문서의 품질을 사전에 필터링



종합 의사결정 매트릭스

12가지 AI 디자인 패턴의 핵심 특징을 비교하고, 프로젝트 요구사항에 맞는 최적의 조합을 선택하기 위한 통합 가이드입니다.



[Problem 1] AI 서비스 조합: 경로 고정 vs 동적 분기

- 1 정적 선형 파이프라인: 고정된 순서로 직렬 실행, 단순·저지연, 유연성 낮음
- 2 중앙 집중형 오케스트레이션: 조건부 분기·재시도·재사용성, 복잡 로직 유리
- 3 자율적 에이전트 루프: 목표지향 탐색, 미정의 절차, 비정형 문제 유연 대응



[Problem 2] AI 서비스 결정: 정적 대상 vs 동적 선택

- 1 정적 엔드포인트 바인딩: 코드/설정에 고정, 오버헤드 無, 변경 시 재배포 필요
- 2 클라이언트 사이드 디스커버리: 레지스트리 조회 후 클라이언트가 선택, 분산 우수
- 3 서비스 브로커/라우터: 중재 계층이 정책 기반 라우팅, 유연·폴백 대응 용이



[Problem 3] AI 서비스 배포: 로컬 처리 vs 중앙 집중

- 1 로컬 엣지 배포: 디바이스 내부에서 완결, 프라이버시 보호, 오프라인·저지연
- 2 중앙 서버 배포: 클라우드 자원 집중 활용, 대규모 모델 운영, 관리·스케일링 유리
- 3 하이브리드 분할 배포: 신뢰도 기반 조건부 엣지-서버 분할, 성능과 비용 균형점



[Problem 4] 컨텍스트 획득: 데이터 보존 vs 토큰 효율성

- 1 전체 이력 버퍼: 원본 100% 가공 없이 보존, 맥락 무결성 보장, 토큰 비용 높음
- 2 압축 요약 기억: 임계치 초과 시 단일 요약본으로 추상화, 토큰 효율적 절감
- 3 세션 계층화 검색: 단기 캐시(L1)와 장기 벡터 스토어(L2) 분리 후 지능형 동적 병합



핵심 Trade-off

단순성 유연성

비용 품질

지연 처리량

일관성 확장성



선택 기준

⚡ 지연시간

💰 비용효율성

🛡️ 신뢰성

🔗 확장성



Part 1 성과 요약 (Completed)

- ✓ **4대 문제 영역 분석:** AI 서비스 조합, 결정, 배포, 컨텍스트 획득에 이르는 핵심 아키텍처 문제 및 과제 정의 완료
- ✓ **12개 디자인 패턴 정립:** 각 패턴의 C&C / Module View 구조, 작동 메커니즘, 장단점 및 실무 적용 시나리오 체계화
- ✓ **통합 의사결정 매트릭스:** 요구사항(지연·가용성·보안·비용 등) 기반의 최적 패턴 선택 기준 및 Trade-off 분석 맵 구축
- ✓ **실무 조합 가이드:** 단일 패턴의 한계를 극복하고 보완하기 위한 하이브리드 다중 패턴 설계 방향 제시



Part 2 여정 예고 (To be continued)

- 5 **[Problem 5] 컨텍스트 획득 효율성:** 데이터 공급 파이프라인 최적화, 지연 시간 최소화 및 컨텍스트 로딩 병목 현상 해결 패턴
- 6 **[Problem 6] AI 서비스 신뢰성:** 입력 가드레일, 출력 검증, 환각 (Hallucination) 방지 및 시스템 안전성 확보를 위한 아키텍처
- 7 **[Problem 7] 수행 효율성:** 로드 밸런싱, 비동기 큐잉, 캐싱 전략을 통한 시스템 처리량(Throughput) 및 응답성 극대화
- 8 **[Problem 8] 모델 진화 지원:** 새도우 배포, A/B 테스트, 점진적 롤아웃 등 무중단 모델 교체 및 지속적 학습 파이프라인

“ 신뢰할 수 있고 확장 가능한 AI 시스템 구축을 위한 8가지 핵심 설계 문제 완전 정복

Part 2에서 새롭고 강력한 아키텍처 패턴들과 함께 여정을 이어갑니다.